Martin Rerabek

ZHAW

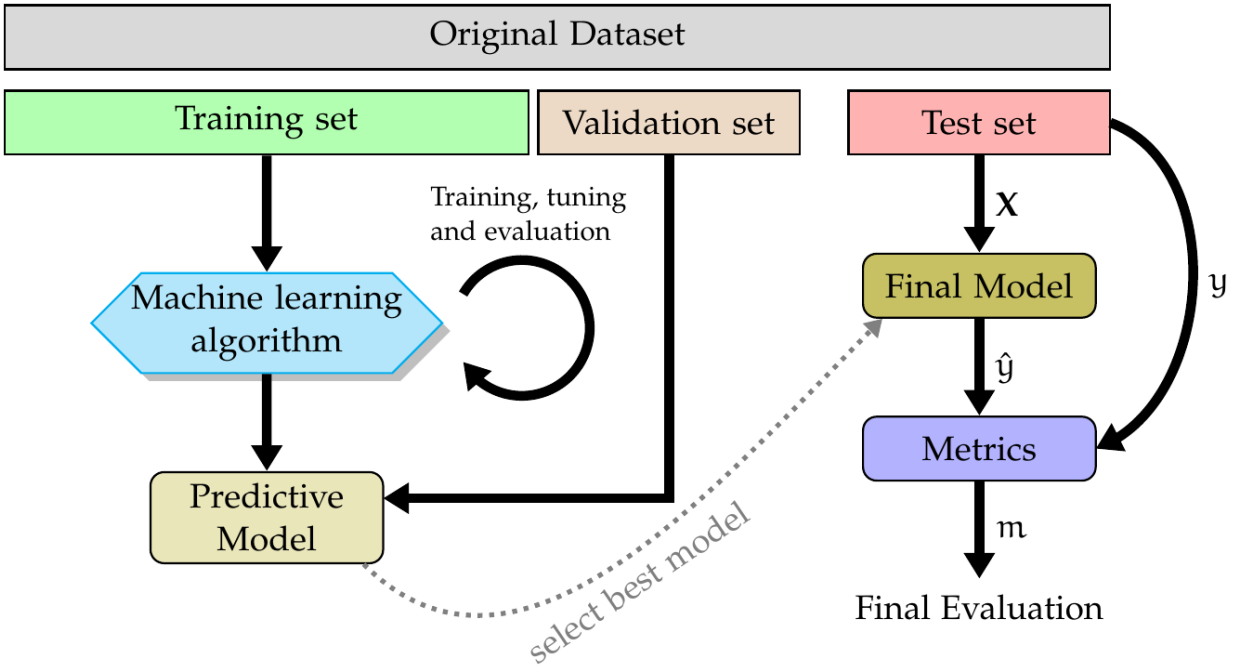
martin.rerabek@zhaw.ch

Week 2

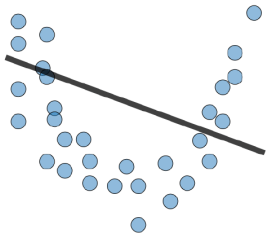
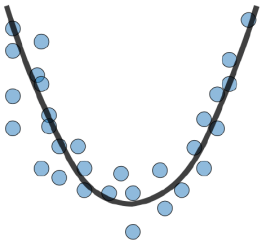
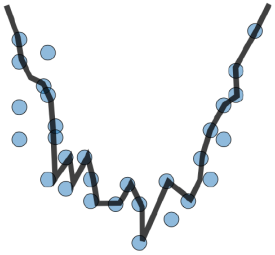
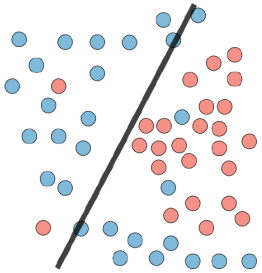
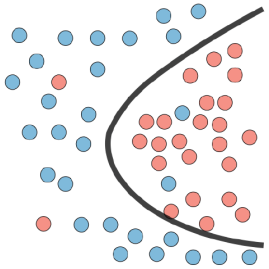
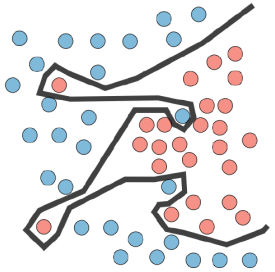
Overview

1. Evaluation of NN
2. Regularization
3. Project Requirements
4. Keras - Tensorflow - Introduction
5. Hands-On
6. DL
7. RNN
8. CNN - teaser

Training NN

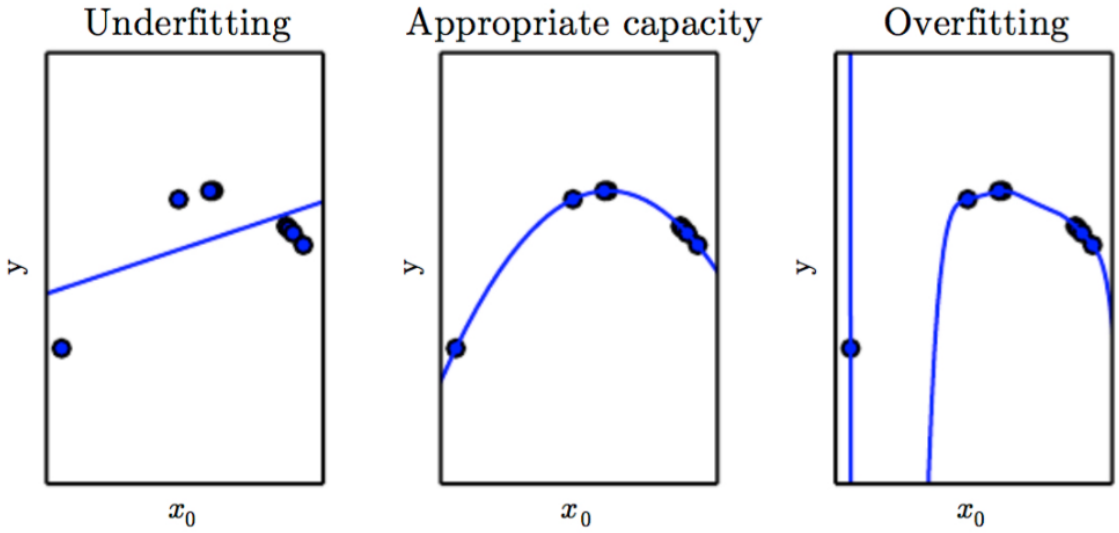

Original Dataset
Training set
Validation set
Test set
Training, tuning
and evaluation
Machine learning
algorithm
Predictive
Model
select best model
x
Final Model
y
ŷ
Metrics
m
Final Evaluation

Training NN

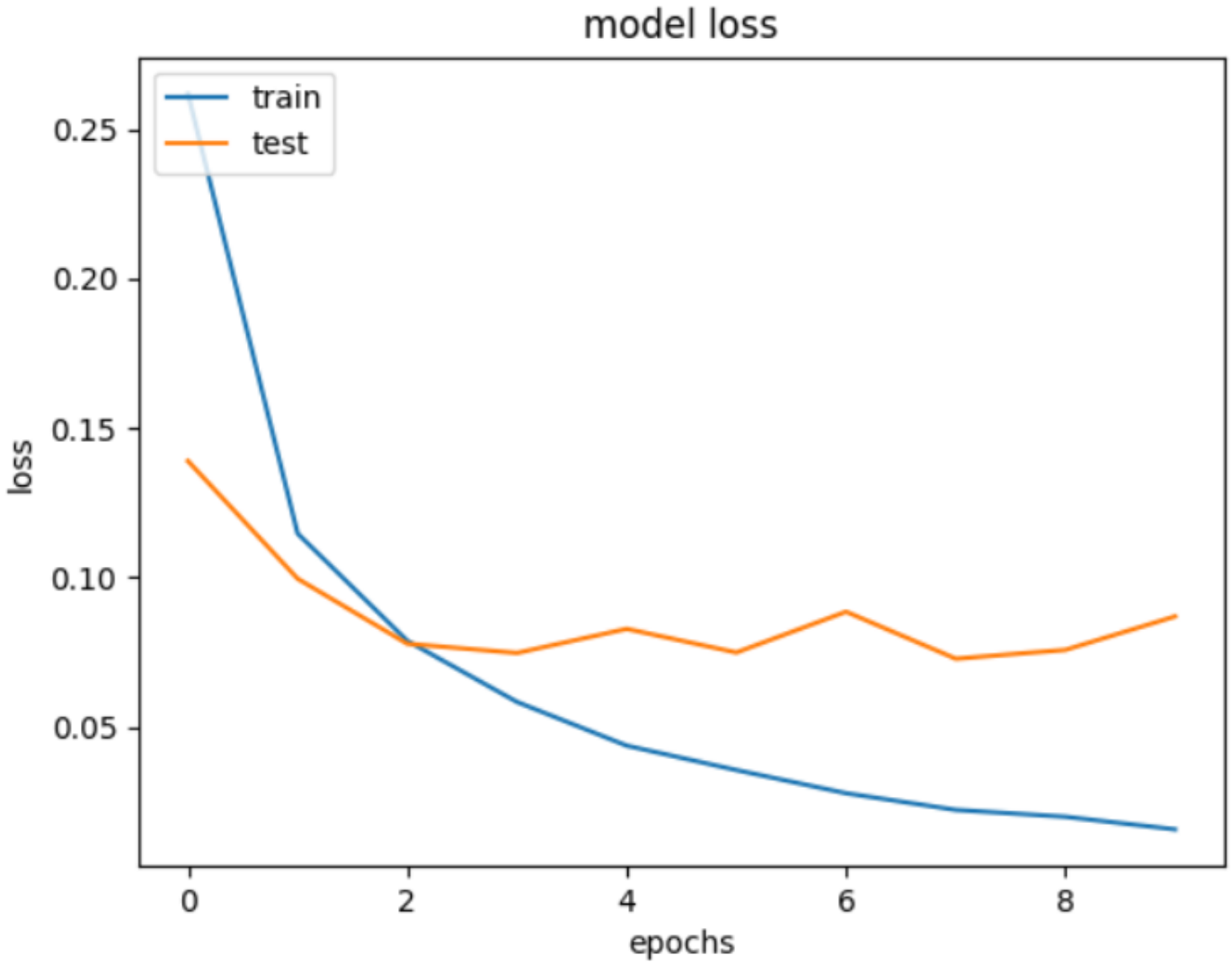
Underfitting	**Just right**	**Overfitting**
- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
		
		
- Complexify model - Add more features - Train longer		- Regularize - Get more data

Model Capacity

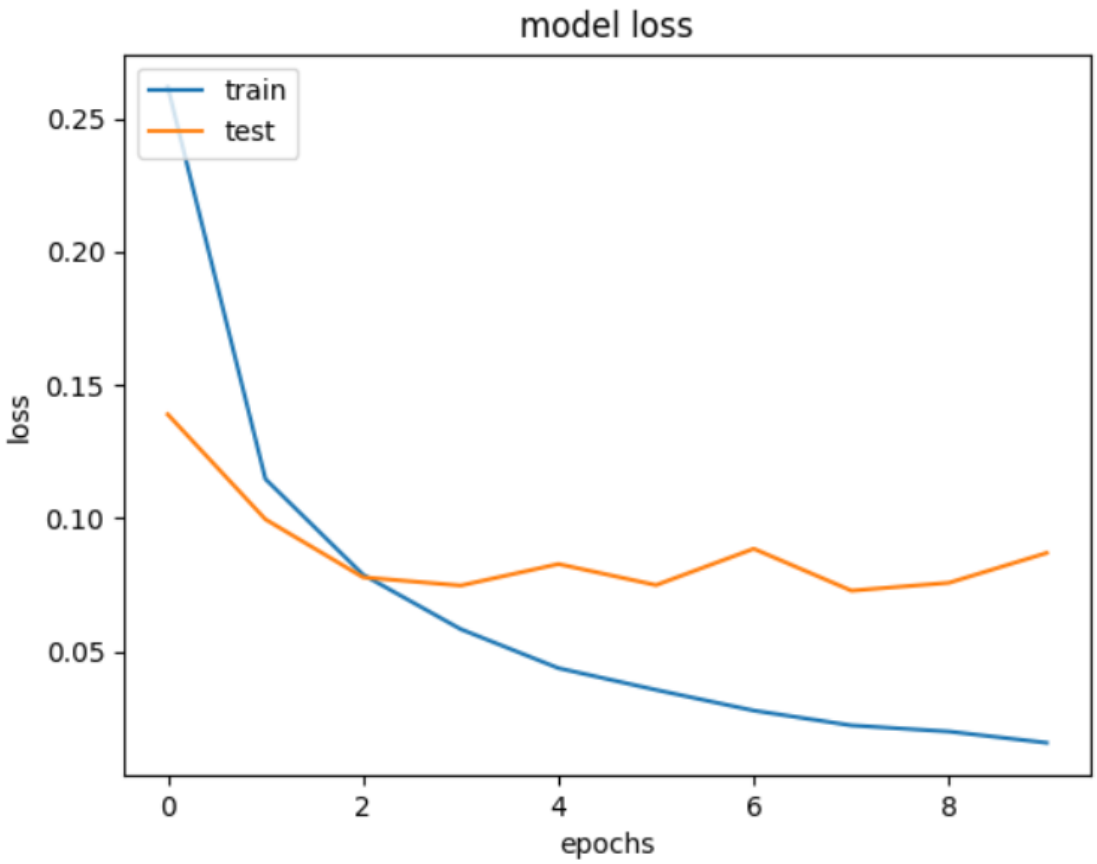
- One way to control model is more likely to overfit or underfit $\rightarrow$ by its capacity
- Model's capacity: ability to fit a wide variety of functions
  - Models with low capacity: struggle to fit the training set
  - Models with high capacity: can overfit by adapting to properties of the training set too much



Training NN



Training NN



- As we can see the training data is learned well, the error on the test however is, after some epochs, not decreasing or even increasing. This is an instance of so-called overfitting.

Training NN

- Performance of algorithm
  1. Make the training error small
  2. Make the gap between training and test error small
- These two factors correspond to two central challenges in neural network learning: underfitting and overfitting
- Underfitting: model is not able to obtain a sufficiently low error value on the training set
- Overfitting: gap between the training error and test error is too large

Regularization

- Central problem in neural network learning: **Generalization**, i.e. to make an algorithm perform well not just on the training data, but also on new, unseen data input
- Regularization: strategies to reduce the test error
- Practical success of an algorithm often hinges on the regularization techniques
- Developing more effective regularization: major research effort in the field.

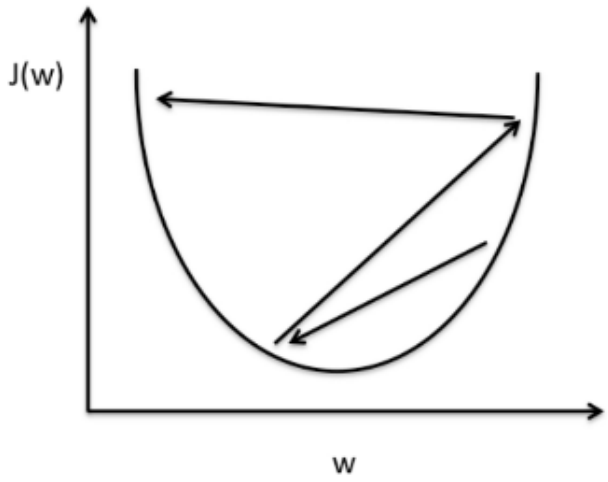
NN Model Complexity

NN Model Complexity

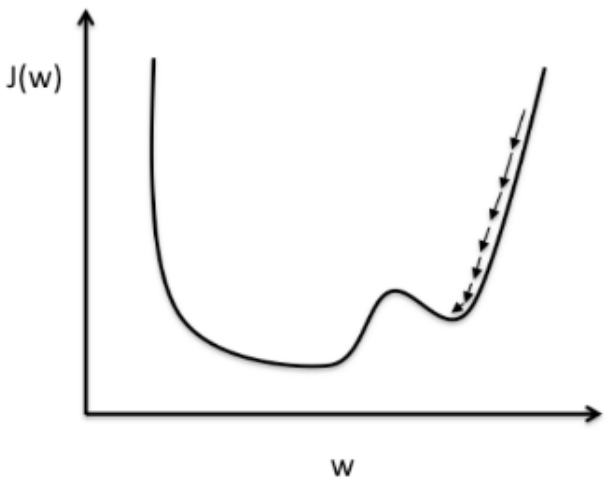
- Number of hidden layers
- Number of neurons within hiden layers
- these increases number of parameters and thus we have more complex model

Learning Rate

- Most models, such as the linear variant of the perceptron, have a learning rate, i.e. a hyperparameter that controls how much we are adjusting the weights in a learning step
- In practice, it often requires some experimentation to find a good learning rate
- A good start is by plotting the cost function for different learning rates



Large learning rate: Overshooting.



Small learning rate: Many iterations until convergence and trapping in local minima.

Weight Regularization

- Given NN architecture and training data
  - Often multiple sets of weight values (i.e. various models) can explain the data or make predictions
  - Simpler models are less likely to overfit than complex ones!
- Limit the complexity of a network by forcing its weights to take only small values - sparse model where some weight close or equal to 0
- This is called weight regularisation and is achieved by adding costs associated with large weights to the loss function of the network - minimize two parts

$$J_w(x) = J_w(x) + \alpha g(\boldsymbol{W})$$

- Two main variations
  - L^1 - $g(\boldsymbol{W}) = ||w||_1 = \sum_i |w_i|$
  - L^2 - $g(\boldsymbol{W}) = ||w||_2 = \sum_i |w_i^2|$ - also reffered to as weight decay

Early Stopping

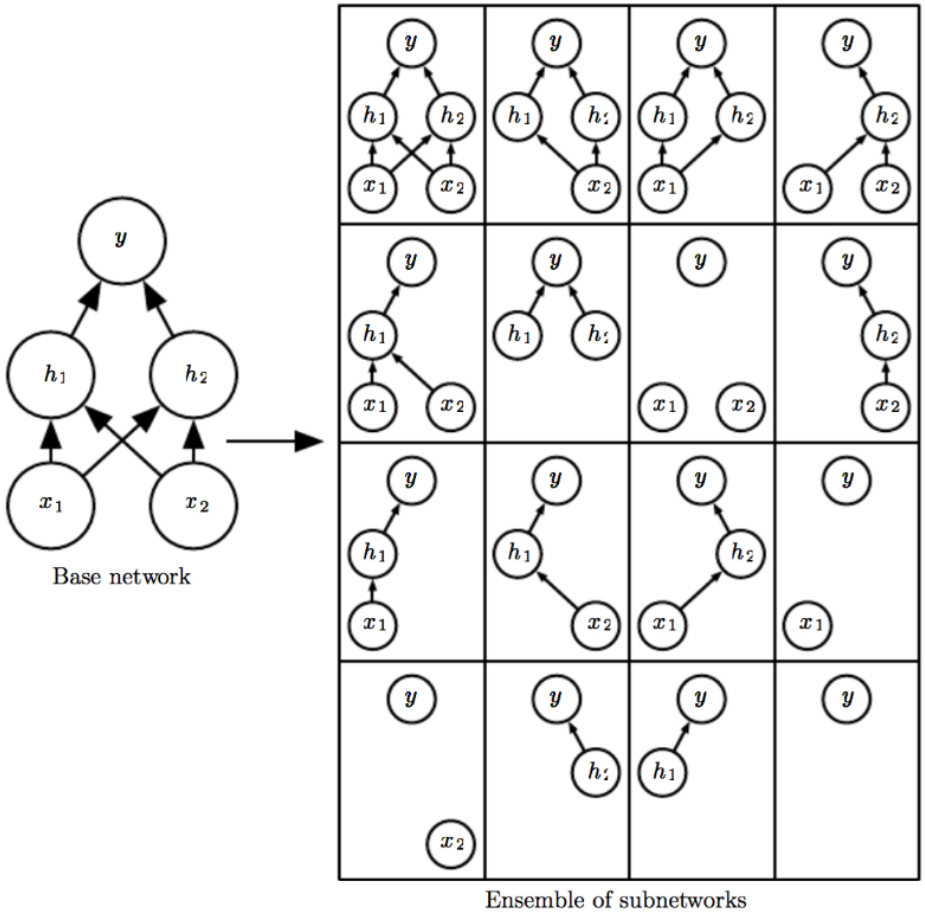
- look at the picture before
- Often one can observe that training error decreases steadily over time, but validation set error begins to rise again
- Better validation set error: parameter setting at the point in time with the lowest validation set error.
- One of the most commonly used form of regularisation in neural networks learning as it can both be effective and simple to implement
- patience parameter in Python

Feature Scaling

- Normalization - features on same scale otherwise model strugles comparing the importance of features
- All features have same importance for the model at the beginning

Dropout I

- Dropout provides a computationally inexpensive but powerful method of regularizing a broad family of models
- Dropout trains an ensemble consisting of all subnetworks that can be formed by removing non-output nodes from an underlying base network



Dropout II

- Dropout - a way to achieve that each hidden node performs well regardless of which other hidden nodes are in the model.
- Dropout thus regularizes each hidden node to be not merely a good feature but a feature that is good in many contexts

SGD

- Size of batch = number of samples of training data processed before updating weights

Criteria	Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
Gradient Computation	Entire dataset at each update	Single random sample per update	Small subset (mini-batch) per update
Update Frequency	Once per epoch	After each sample	After each mini-batch
Computation Cost per Update	High (proportional to dataset size)	Low (one sample)	Moderate
Memory Usage	High	Low	Moderate
Convergence Speed	Slow	Fast but noisy	Balanced
Stability of Convergence	Stable, smooth trajectory	High variability, noisy updates	Less noisy than SGD
Generalization Ability (Overfitting)	Risk of overfitting	Better generalization due to noise	Balanced risk
Parallelization	Difficult	Easy	Easier than batch

Data Augmentation

- One of the most crucial method - get more data
- Collect, synthesize, augment-pre-postprocess - i.e. image augmentation

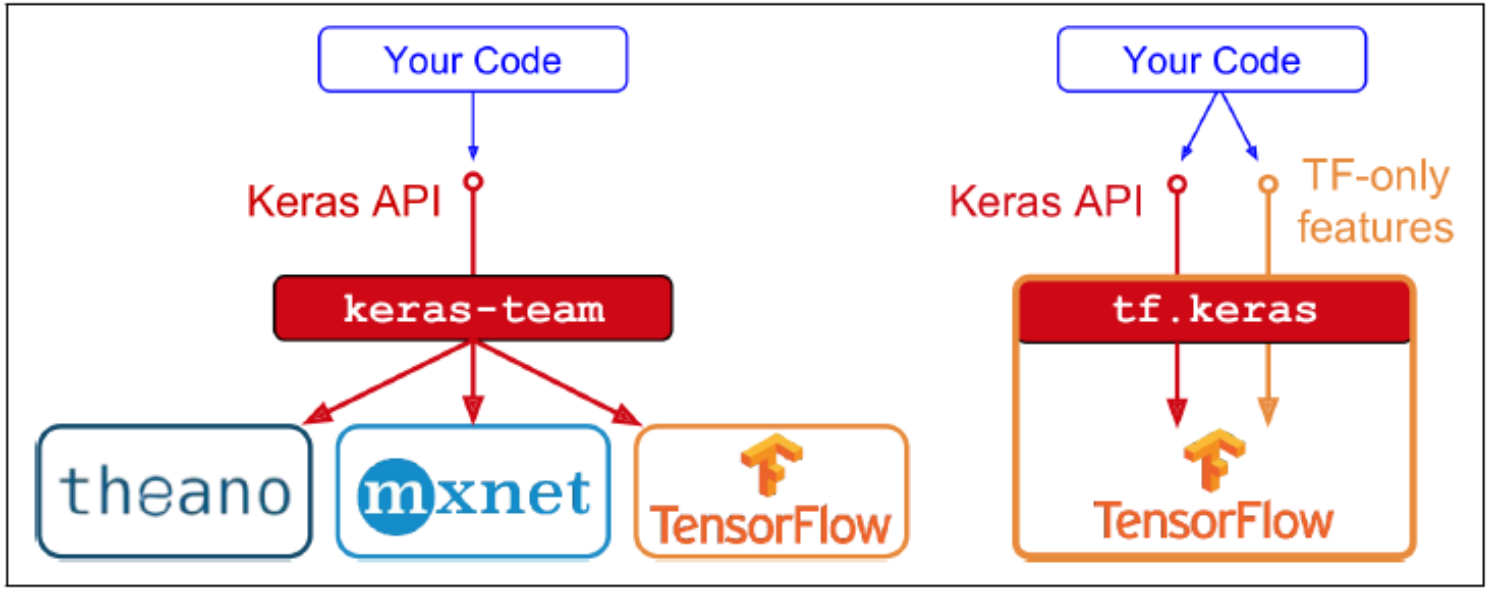
Data Augmentation Techniques for Different Data Types

- **Image Data Augmentation**
  - Rotation, Flipping, Cropping, Scaling, Color Jittering
- **Text Data Augmentation**
  - Synonym Replacement, Random Insertion, Random Deletion, Back Translation
- **Audio Data Augmentation**
  - Noise Injection, Time Shifting, Pitch Shifting, Speed Variation
- **Tabular Data Augmentation**
  - SMOTE (Synthetic Minority Over-sampling Technique), Random Sampling, Feature Perturbation

Project Requirements I

- Explain task to be solved
- Analyse your data, visualize and explain data features
- Explain your approach - model, architecture, optimizer, parameters
- Experiment with your model, change it, i.e. tune hyperparameters
- Show/explain/visualize your results
- Point out problems - ideas for next steps
- Presentation take 15-20min per group
- Groups ...

Keras - Tensorflow I



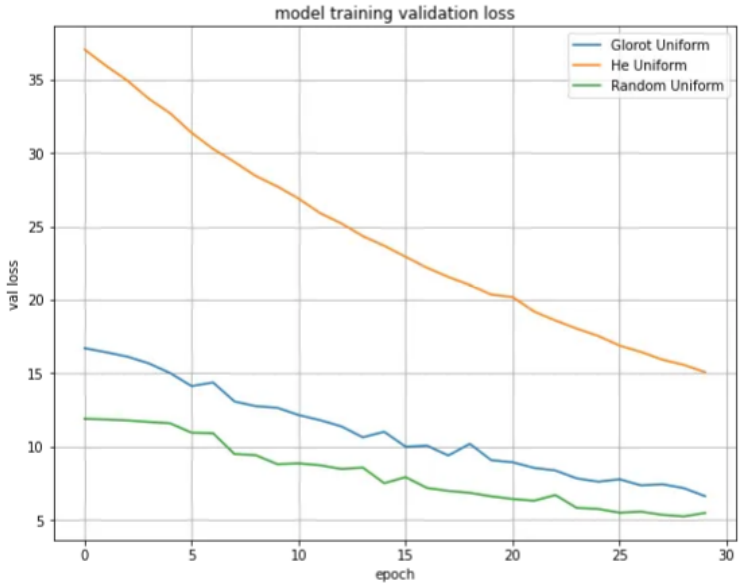
- Keras is a high-level Deep Learning API that allows you to easily build, train, evaluateand execute all sorts of neural networks
- One can choose from three popular opensource deep learning libraries: TensorFlow, Microsoft Cognitive Toolkit (CNTK) or Theano

Keras - Tensorflow II

- Moreover, since late 2016, other implementations have been released. You can nowrun Keras on Apache MXNet, Apple's Core ML, Javascript or Typescript (to run Kerascode in a web browser), or PlaidML (which can run on all sorts of GPU devices, not just Nvidia).
- Moreover, TensorFlow itself now comes bundled with its own Keras implementation called **tf.keras**
  - Supports only TensorFlow as the backend, but it hasthe advantage of offering some very useful extra features - TensorBoard
  - it supports TensorFlow's Data API which makes it quite easy to load and preprocess data efficiently
  - we will use tf.keras
  - allow to use TensorFlow-specific features

Weights-Optimizers-Parameters Where to start? I

- Weights initialization
  - Zeros, Random vs Glorot or He where weigths drawn from standard distribution



Weights-Optimizers-Parameters Where to start? II

- Optimizers
  - Stochastic Gradient Descent (SGD)
    - Updates parameters using the gradient of the loss
    - Updates $\approx$ batch size parameter
    - For large datasets due to memory efficiency
  - RMSprop (Root Mean Square Propagation)
    - Adapts learning rates based on recent gradient magnitudes, helping to mitigate both vanishing and exploding gradients.
    - Recurrent neural networks (RNNs), Non-stationary objectives, comoplex functions
  - Adam (Adaptive Moment Estimation)
    - Combines advantages of above, providing fast convergence, stability, and computational efficiency
    - Robust optimizer for most problems.

Weights-Optimizers-Parameters Where to start? III

- What default parameters to use?
  - learning rate - 0.01, how many epoch, what batch size, size of hiden layers
  - techniques, such as callbacks, hyperparameter tuning - later with DL

Important Activation Functions I

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- Softmax
- Swish

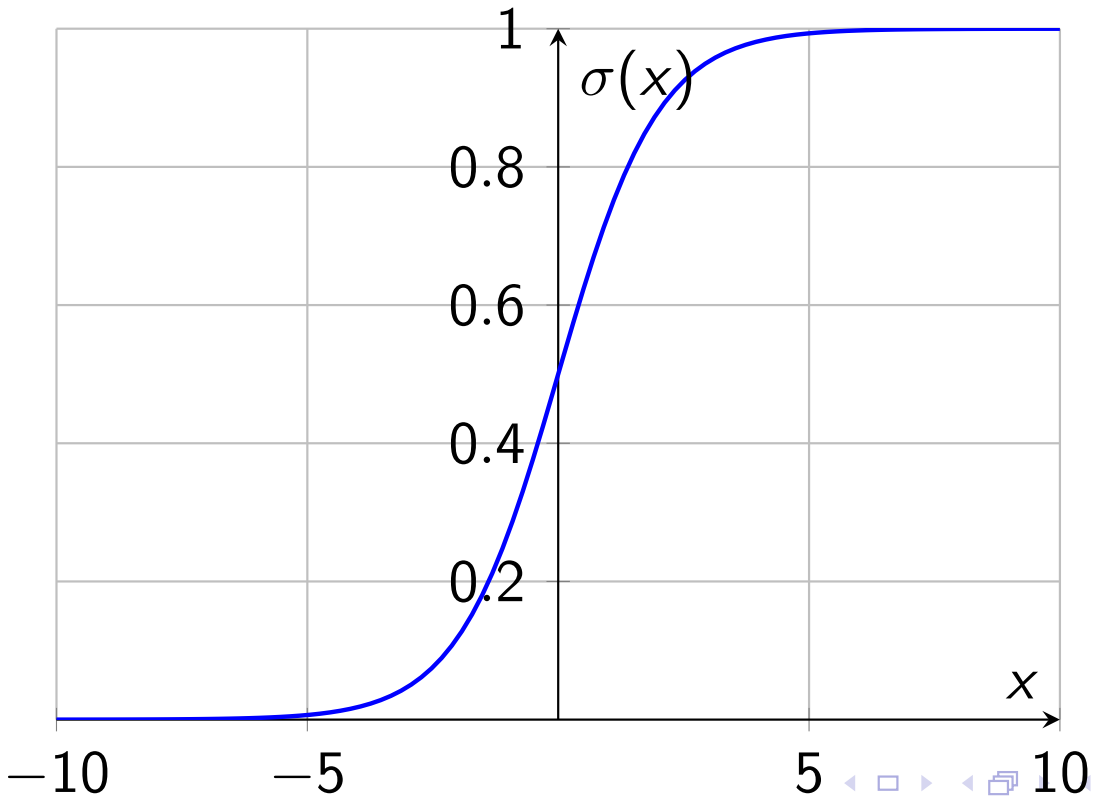
Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

Use Case: Binary classification problems.

Pros: Smooth gradient, output values bound between 0 and 1.

Cons: Can cause vanishing gradient problems, slow convergence.



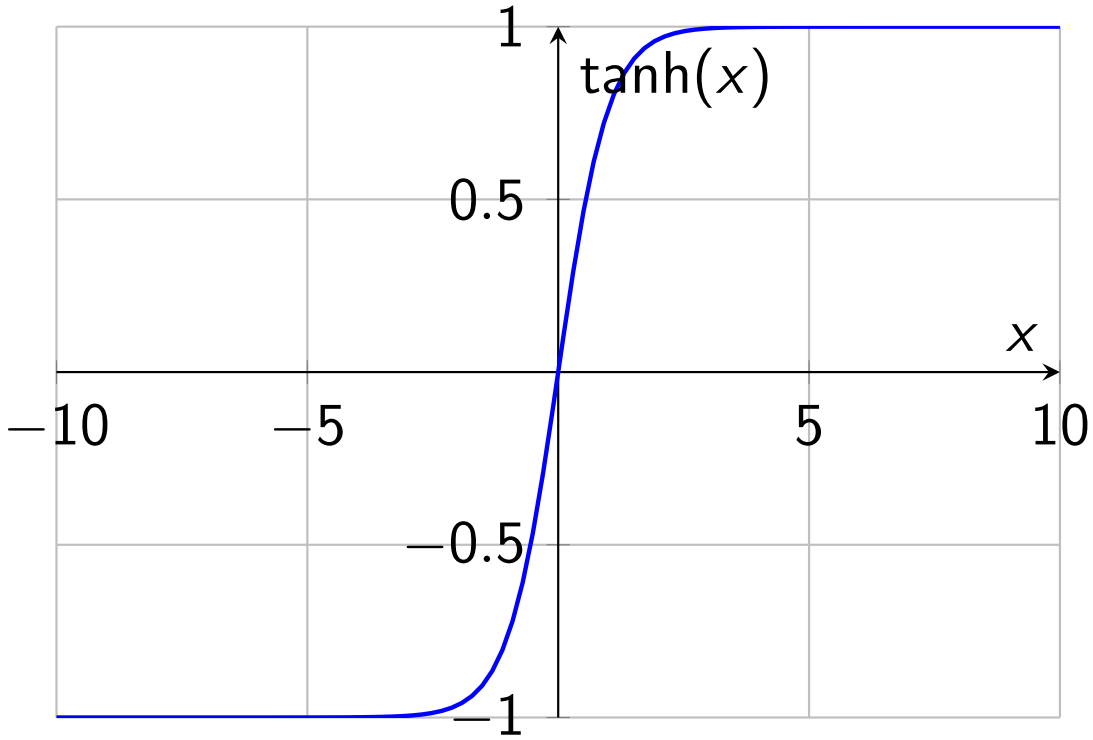
Tanh Activation Function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{2}$$

Use Case: Hidden layers of neural networks.

Pros: Zero-centered output, which can make optimization easier.

Cons: Can also suffer from vanishing gradient problems.

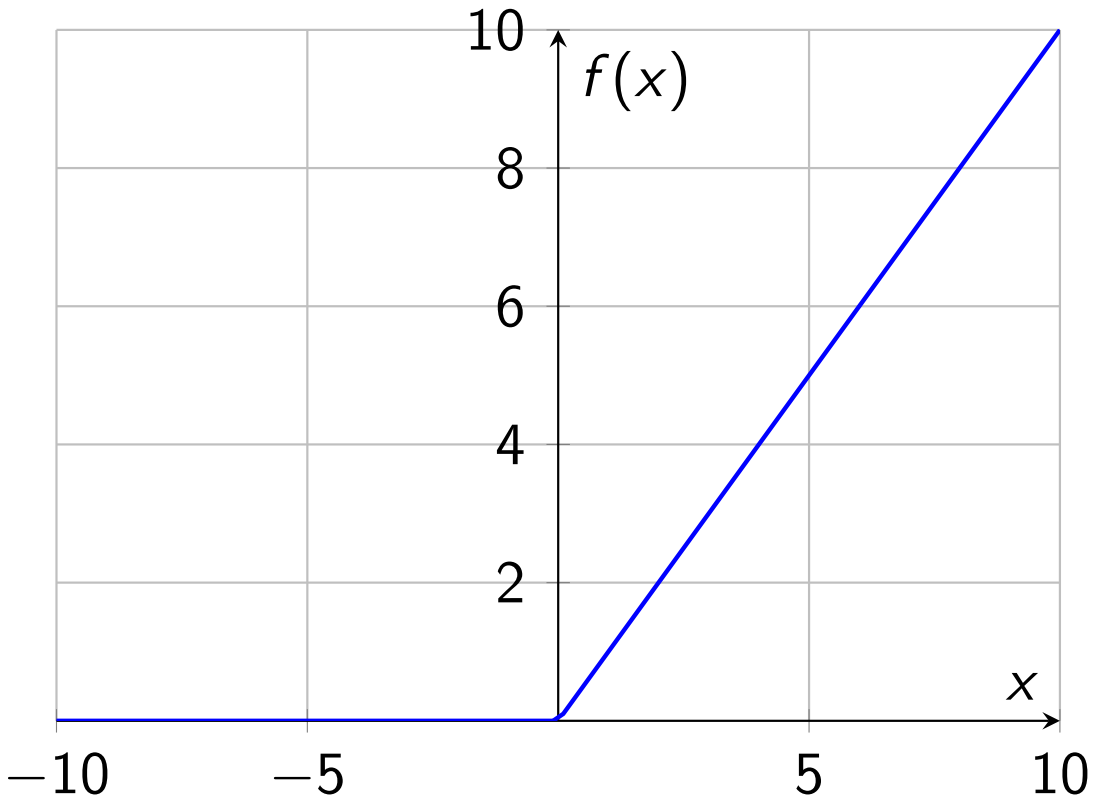


ReLU Activation Function I

$$f(x) = \max(0, x) \tag{3}$$

Use Case: Widely used in hidden layers of neural networks.
Pros: Efficient computation, mitigates vanishing gradient problem.
Cons: Can cause "dying ReLU" problem where neurons stop activating.

ReLU Activation Function II



ReLU Activation Function III

The "dying ReLU" problem in neural networks occurs when neurons using the (ReLU) activation function become inactive and consistently output zero. This happens because the ReLU function outputs zero for any negative input, and if a neuron's input remains negative, it stops learning as its gradient becomes zero during backpropagation

- **Definition:** Neurons using ReLU activation become inactive, consistently outputting zero.
- **Causes:**
  - Poor weight initialization
  - Unbalanced data
- **Consequences:**
  - Learning impediment
  - Inefficient gradient descent
- **Mitigation:**
  - Proper weight initialization (e.g., He initialization)
  - Use of Leaky ReLU

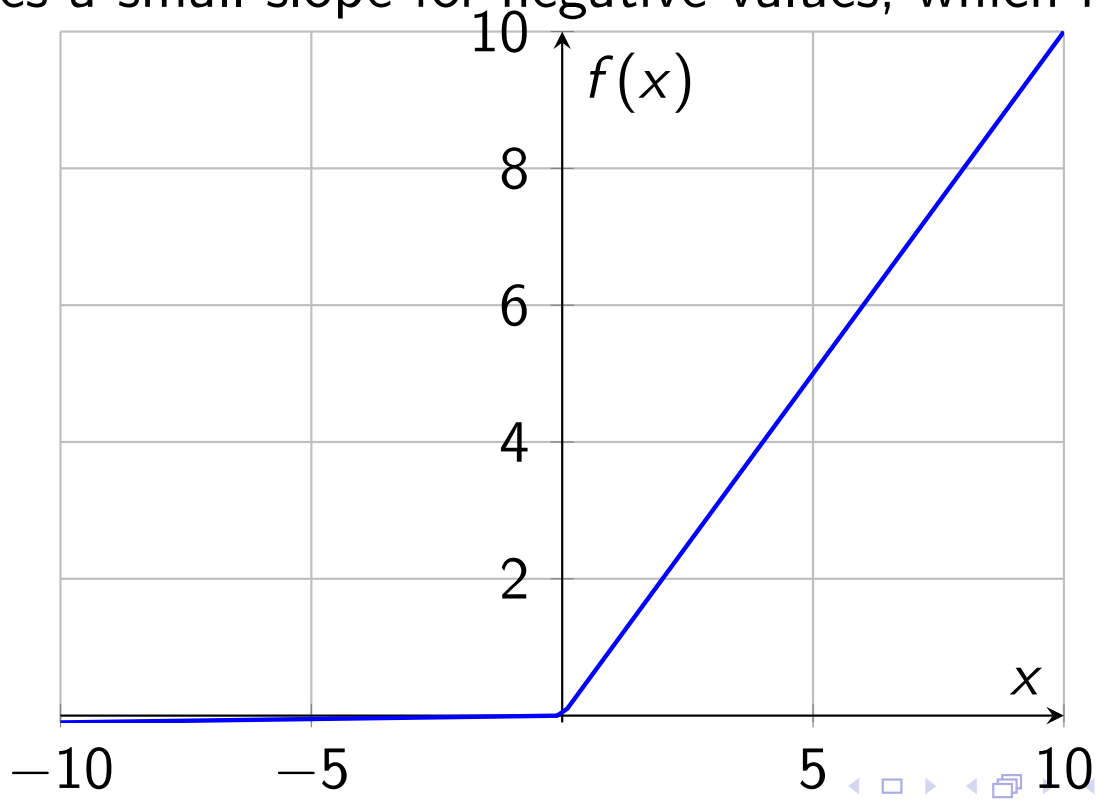
Leaky ReLU Activation Function

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{4}$$

Use Case: Used to address the "dying ReLU" problem.

Pros: Allows a small gradient when the unit is not active.

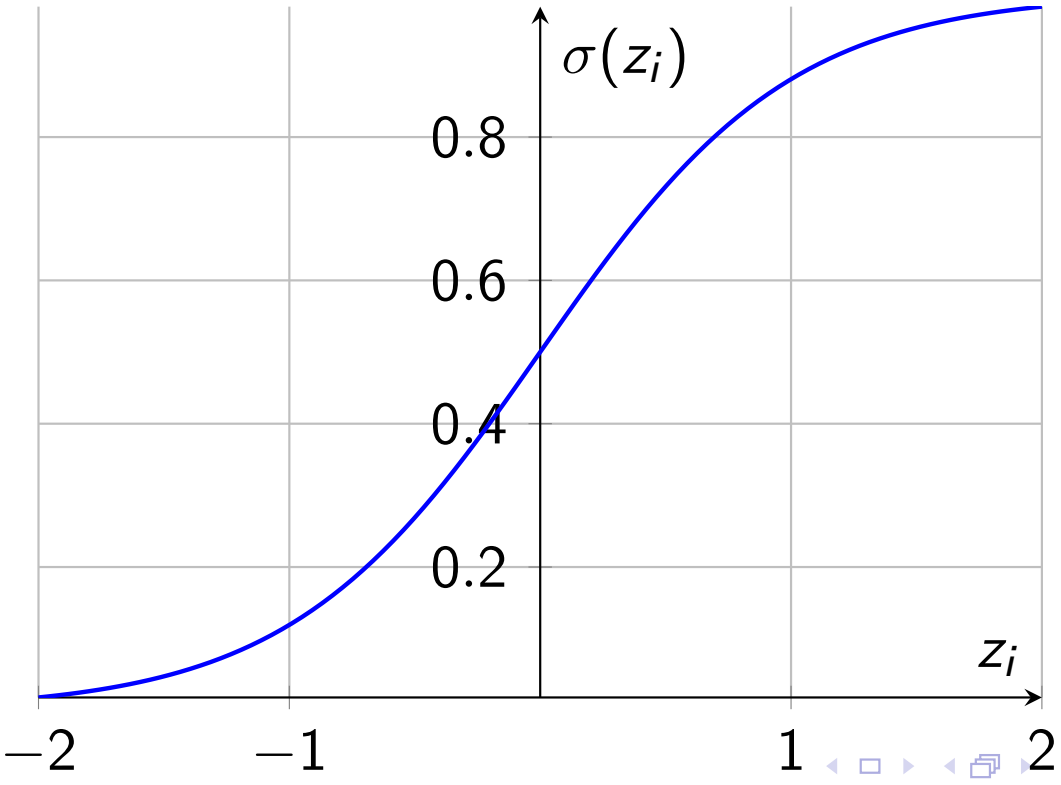
Cons: Introduces a small slope for negative values, which may not always be beneficial.



Softmax Activation Function

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{5}$$

Use Case: Output layer for multi-class classification problems.
Pros: Converts logits to probabilities, useful for classification.
Cons: Computationally expensive for large number of classes.



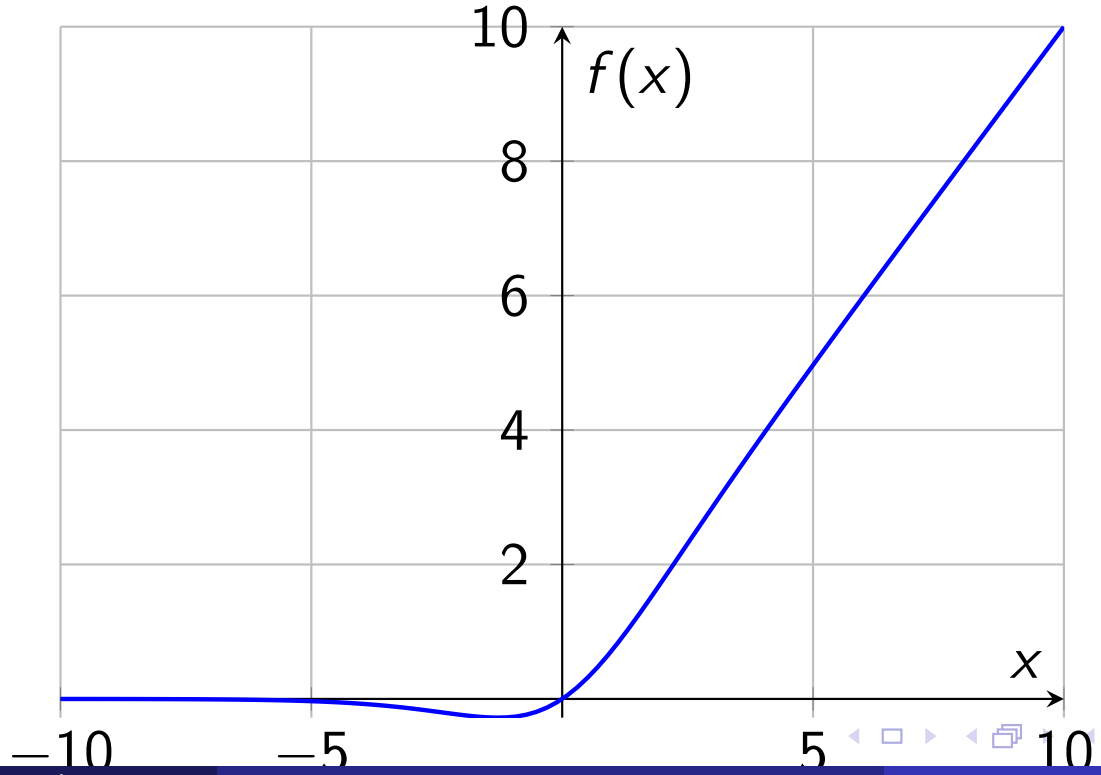
Swish Activation Function

$$f(x) = x \cdot \sigma(x) \tag{6}$$

Use Case: Used in various deep learning models for improved performance.

Pros: Smooth and non-monotonic, can outperform ReLU in some cases.

Cons: More computationally intensive than ReLU.



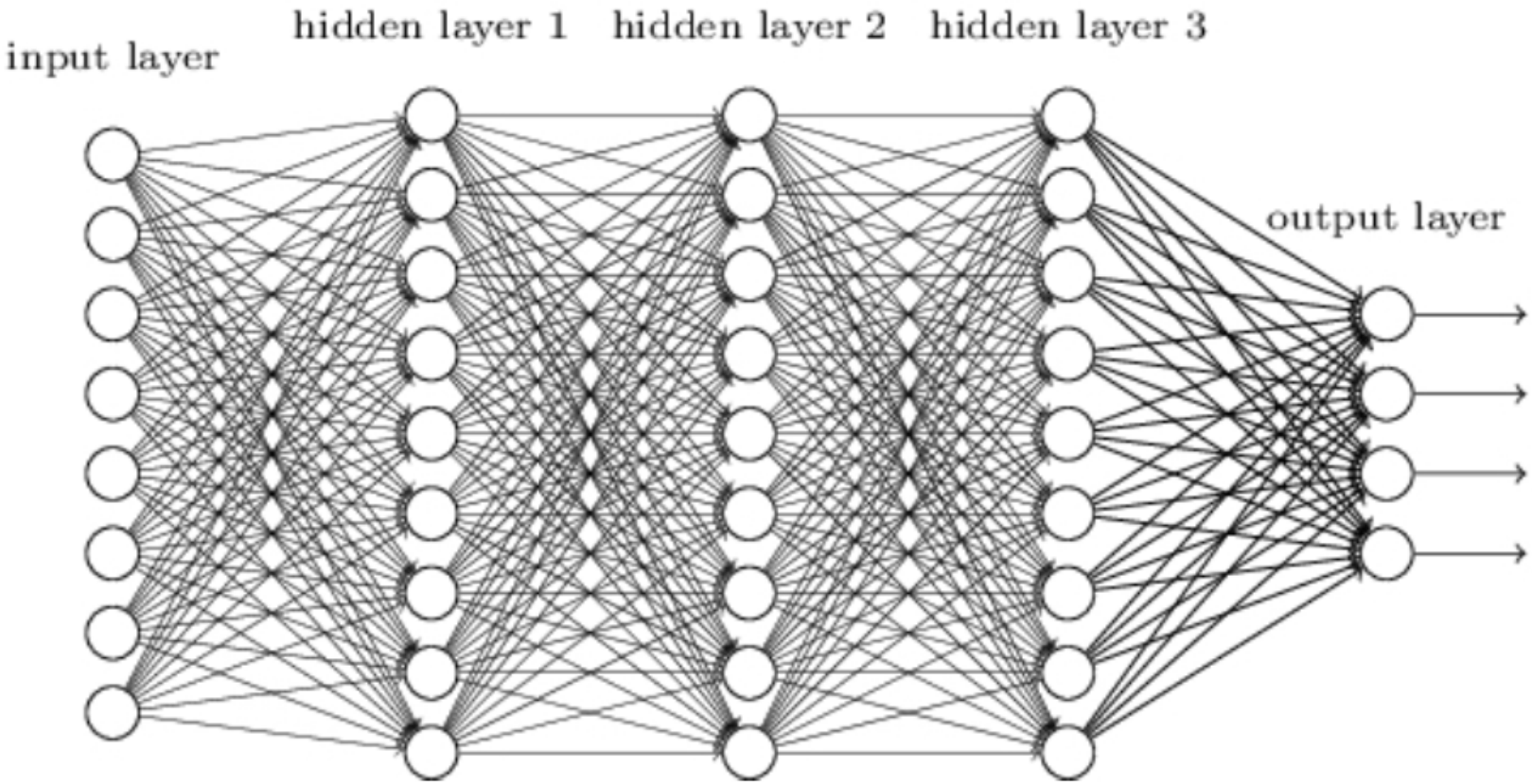
Hands-On Session

Training FNN

- Example on MNIST dataset
- Example on structured dataset
- Group work - train FNN on boston housing dataset

Deep Learning

- Deep learning network (DNN): artificial neural network (ANN) with many hidden layers.



Depth and Complexity

- **Multiple Layers**: A DNN consists of multiple hidden layers between the input and output layers. Each layer extracts different features from the input data, allowing the network to learn more abstract and complex representations as the data passes through each layer.
- **Hierarchical Learning**: Early layers in a DNN typically learn simple features (like edges in an image), while deeper layers combine these simple features to form more complex patterns (like shapes or objects). This hierarchical learning enables the network to understand and process intricate data structures.

Power of Multiple Layers

- **Feature Abstraction**: Each layer in a DNN can be seen as a transformation of the input data. The deeper the network, the more abstract the features it can learn. This abstraction allows the network to generalize better and perform well on complex tasks.
- **Referencing Earlier Layers**: Later layers in the network can refer back to the outputs of earlier layers. This means that the network doesn't just encode the input variations directly but builds upon the learned features from previous layers to make more informed decisions.

Practical Implications

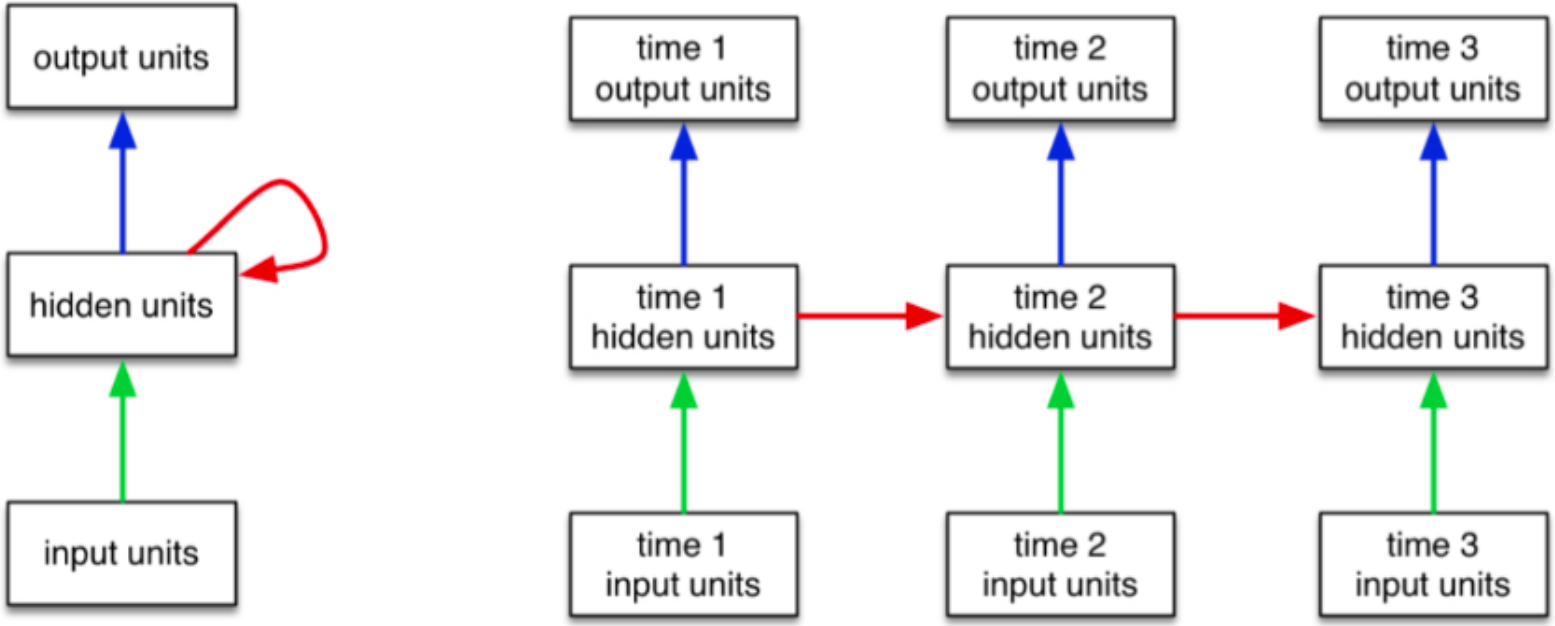
- **Complex Tasks**: The depth of DNNs makes them particularly powerful for tasks like image recognition, natural language processing, and speech synthesis, where understanding complex patterns and relationships in the data is essential.
- **Improved Performance**: Deeper networks have been shown to achieve higher accuracy and better performance on a wide range of tasks compared to shallower networks.

Recurent Neural Networks I

- FNN: information flows through the network in one direction
- FNN with feedback connections: recurrent neural networks (RNN)
- RNN can be viewed as dynamic systems, i.e., systems that change over time. There is a set of input units, hidden units, and output units, and the hidden units feed into themselves.
- The self-loops mean that the values of the hidden nodes at one time step depend on the values at the previous time step.

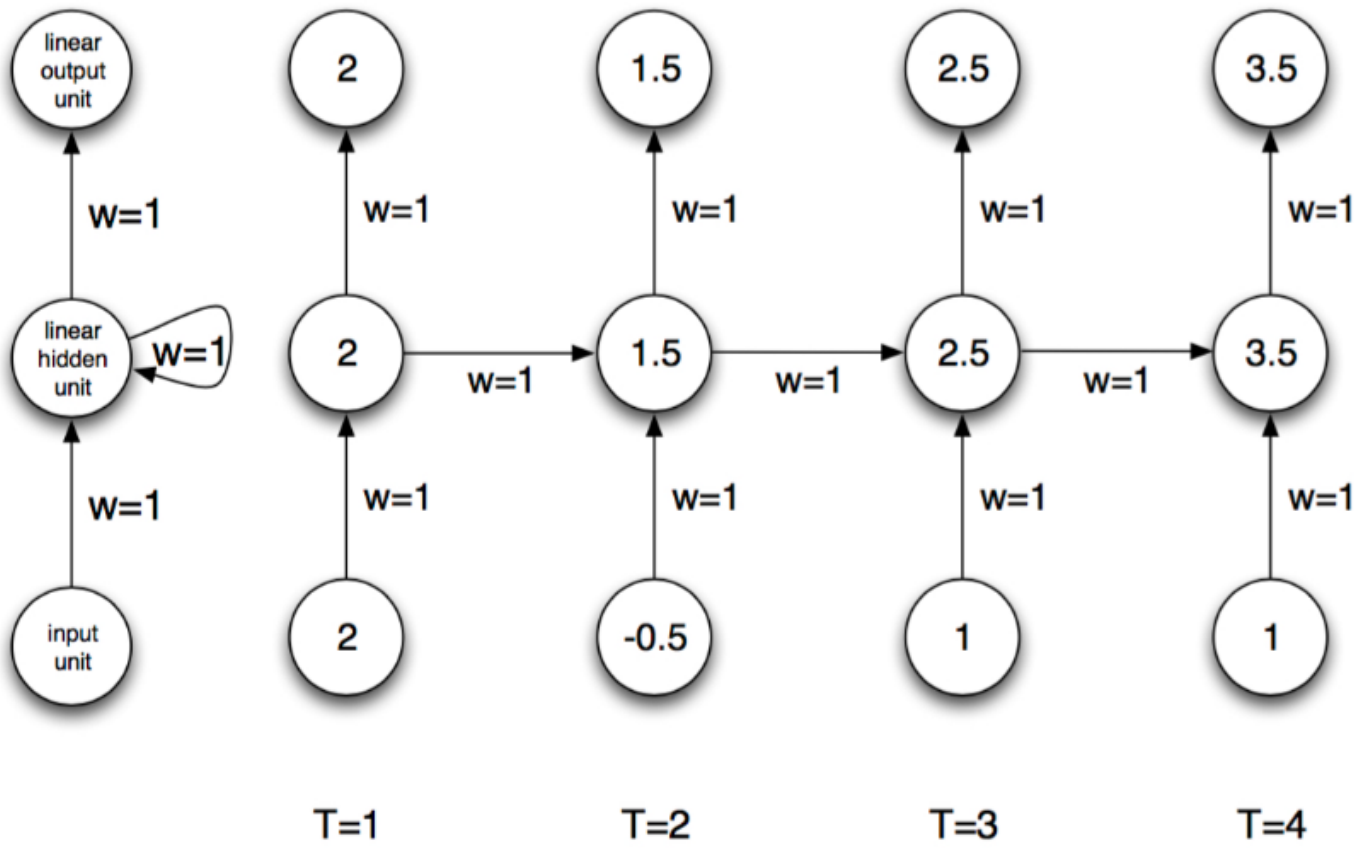
Recurent Neural Networks II

- The computation that RNN is performing can be better understood by unrolling the network, i.e., by explicitly representing the nodes at all time steps, as well as the connections between them
- For a given sequence length, the unrolled network becomes essentially a feedforward neural net, where the weights are shared between the time steps



Recurent Neural Networks III

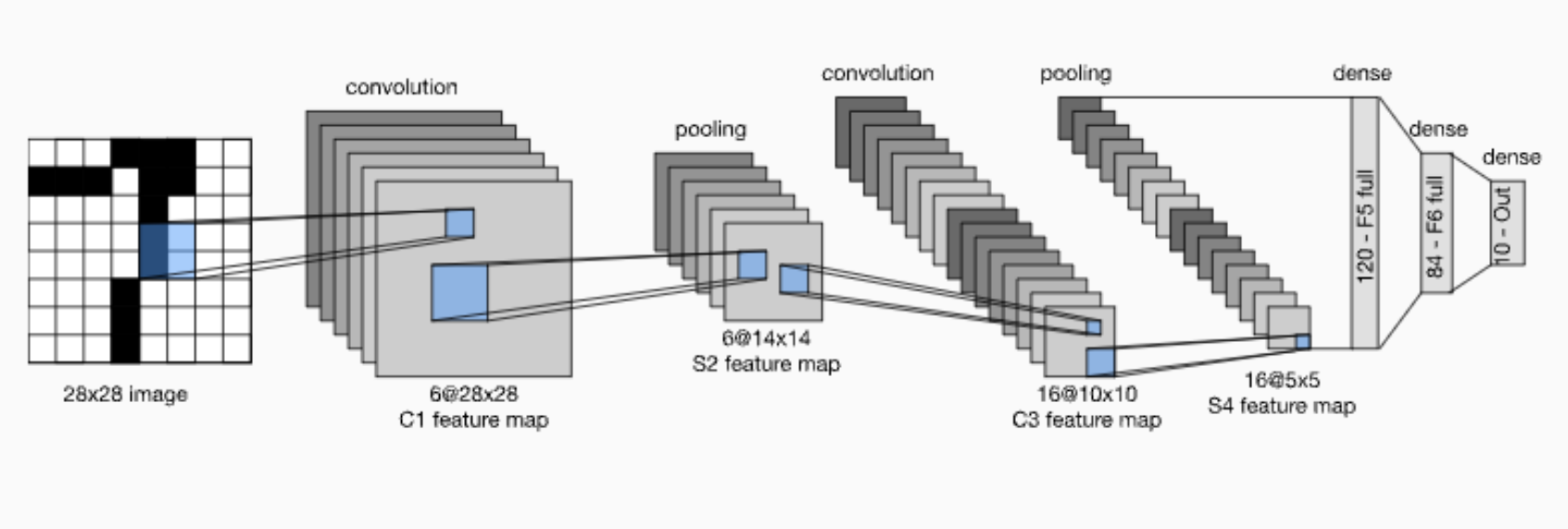
- RNN example - summing inputs over time
- All activation functions assumed linear, $f(z) = z$



Recurent Neural Networks IV

- In these examples, the weights and biases are set by hand in order to compute a specific function
- We can also learn the weights with "backpropagation in time". The basic idea is to run the ordinary backpropagation algorithm on the unrolled graph
- RNN are especially useful for sequence-to-sequence prediction, i.e., tasks where inputs and outputs are both sequences. Typically, in sequences causality and order matter - RNN can exploit such temporal ordering of data points
- RNN can be very powerful
- However, because there are potentially very deep, the gradient descent updates may become unstable because the derivatives explode or vanish over time.
- There are various ways to deal with this problem - explained later

LeNet


convolution
pooling
convolution
pooling
dense
dense
dense
28x28 image
6@28x28
C1 feature map
6@14x14
S2 feature map
16@10x10
C3 feature map
16@5x5
S4 feature map
120 - F5 full
84 - F6 full
10 - Out